Actividad: “Enfoque Modular y Diseño Top-Down”

E.E.S.T N°1 de Vicente López

Curso: Laboratorio de Programación 5°3°

Alumno: Benjamin Hlynczuk

Profesor: Ganduglia Yamil & York Elias Mansilla Muñoz

Introducción

El objetivo de la actividad es aplicar un enfoque estructurado y Top-Down en C++, dividiendo un problema en módulos más pequeños mediante el uso de funciones.

Para ello desarrollé un programa que calcula el área de un círculo a partir de la fórmula $A = \pi \cdot r^2$. El programa solicita el radio al usuario, valida que sea un valor positivo y delega el cálculo en una función propia, calcularAreaCirculo, declarada mediante un prototipo previo a main(). Dentro de esa función, el valor de π se definió como una constante (const) para evitar que sea modificado durante la ejecución. Además, se incorporó una simulación en ASCII de una diana de tiro cuyo tamaño depende del radio ingresado.

El código fuente se compiló con g++ y se organizó en un repositorio de GitHub con la estructura de carpetas solicitada (src/, docs/ y capturas/), junto con el diagrama de flujo, el archivo README y las capturas de evidencia. Por último, el informe incluye las respuestas al cuestionario técnico de control sobre el prototipo de función, el filtro de consistencia y la protección de constantes.

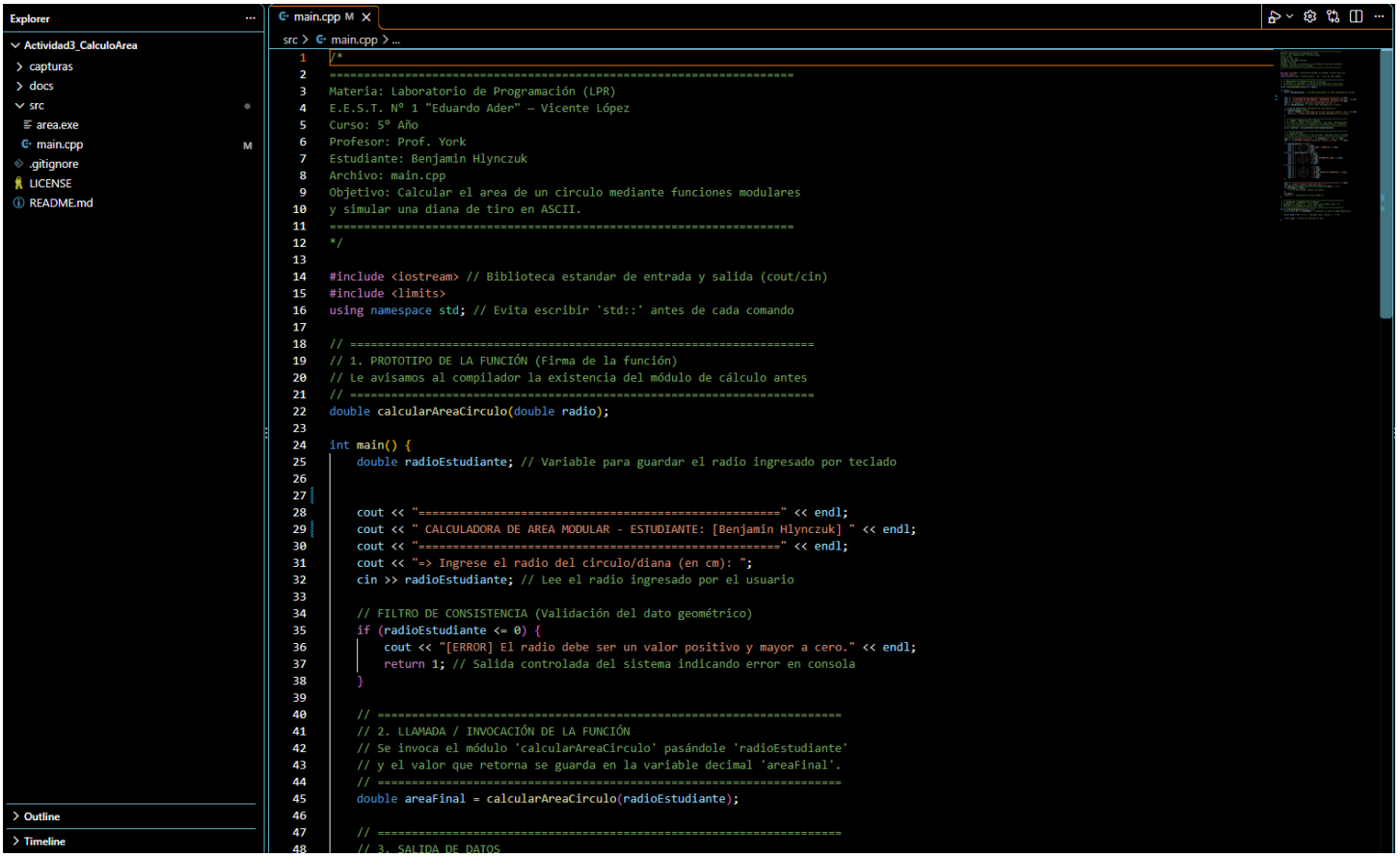
Desarrollo

El desarrollo de la actividad comenzó con la creación de un repositorio local vacío, allí, creó las carpetas y archivos (por ahora, en blanco) pedidos por el profesor. Una vez preparado el repositorio, lo abrí en Visual Studio Code y pegué el código proporcionado para empezar a trabajar en él.

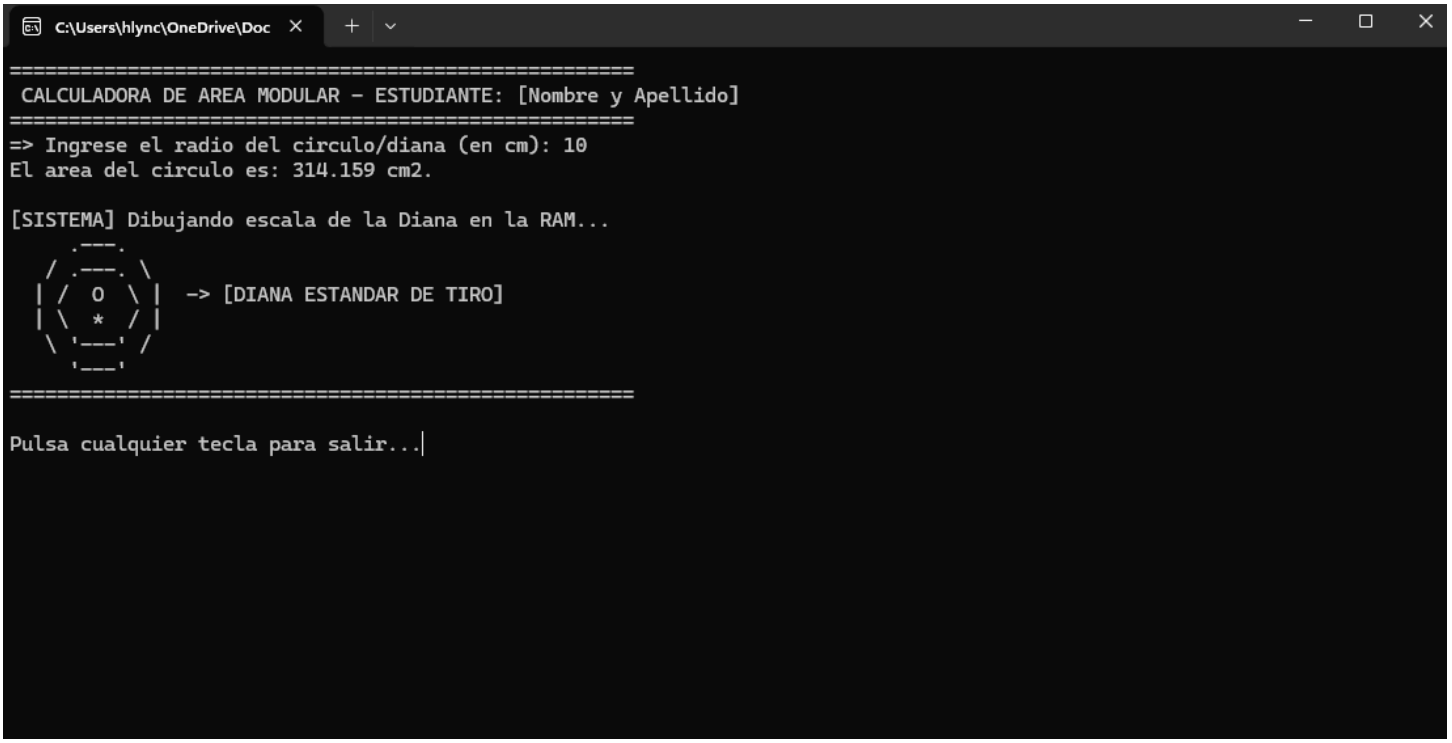
En la primera sección, se debía llamar a la función que calcula el área de un círculo para asignarle el valor que retorna a una variable llamada ‘areaFinal’. Para hacer esto, primero debia crear la variable, a esta le asigne el tipo “double”, ya que era necesario que tenga al menos doble de precisión que un float (15 decimales), luego, la iguale con el valor que retorna la función ‘calcularAreaCirculo(radioEstudiante)’.

En el segundo bloque, lo único pedido era hacer el código para la salida del valor de areaFinal en la terminal, para hacerlo, simplemente utilice un ‘cout’ con texto en conjunto al valor de ‘areaFinal’. El texto de salida sería: “El area de circulo es: ” + areaFinal + “cm2.”

Al tercer y último bloque, se le debía agregar la lógica detrás de la función “calcularAreaCirculo(double r)”. Lo que hice yo fue crear un ‘double’ constante que tenga el valor de pi, a este valor, se lo multiplique dos veces por r (valor extraído de lo que el usuario puso), así simulando pi . r^2, devolviendo el resultado.



Captura del codigo fuente



Captura del ejecutable

Cuestionario tecnico de control

Pregunta 1: El compilador de C++ lee el archivo de arriba hacia abajo. En el programa, la función calcularAreaCirculo está definida después de main(). Si se borra el prototipo, cuando el compilador llega a la llamada dentro de main() todavía no sabe que esa función existe, ni qué tipo de dato devuelve, ni qué parámetros recibe. Por eso no puede verificar si la llamada es correcta y muestra un error del tipo “la función no fue declarada en este ámbito”. El prototipo funciona como un aviso previo que le da esa información antes de que empiece a leer main().

Pregunta 2: Geométricamente, un círculo de radio 0 cm es solo un punto y su área es 0, así que no representa una figura real. Un radio de -5 cm no tiene sentido físico, porque el radio es una longitud y no puede ser negativa. El problema es que, como la fórmula eleva el radio al cuadrado, el programa devolvería un área positiva (78,54 cm²) para un círculo que no existe, y el usuario no se daría cuenta del error. Controlar esto es importante porque un dato inválido que no se detecta genera resultados incorrectos que pueden arrastrarse a otros cálculos del sistema. Validar la entrada evita esos errores silenciosos.

Pregunta 3: Declarar un valor como `const` hace que el compilador impida modificarlo en cualquier parte del programa. En un sistema crítico, como el de un avión o un equipo médico, un valor que cambia por accidente (un error de programación o una línea mal escrita) puede producir cálculos erróneos con consecuencias graves. Con `const`, ese error se detecta al compilar y no cuando el sistema ya está funcionando. Además, deja claro para cualquier persona del equipo que ese valor es fijo, lo que mejora la legibilidad y el mantenimiento del código.

Conclusión

En esta actividad, trabajé con el modelo Top - Down funciones dentro del código. El modelo sirve para dividir secciones del codigo, trabajando cada parte por separado, las funciones nos sirven mucho para realizar esto, ya que podemos llamarlas en cualquier momento y asignarles secciones del código.

Referencias:

Repositorio Github: Link

Laboratorio de Programación 5°3°

Actividad: 🛠️📲💻 ACTIVIDAD 3: ENFOQUE MODULAR Y DISEÑO TOP-DOWN - LPR 5° 3° A-…